DPO loss core and a real DPO run

Part 1 · Systems Architect — Rafid Mohammad Ahsan · file `01_DPO_Loss_Core.py` · run: `python 01_DPO_Loss_Core.py`

Pure PyTorch `get_batch_logps` (vectorised `torch.gather`, `-100` masking, optional length averaging) and `DPOLoss` (log-ratio deltas, `logsigmoid` for stability). Tested first on random logits, then used to train the AURA Lab model (`aura_lab.py`) for 100 steps on honest ≻ sycophantic and confirm ≻ contrarian pairs. Checks: start loss equals ln 2 (policy = reference); after `backward()` all 41 policy tensors have gradients and all 41 reference tensors have none. Held-out honest correction rises from 48.5 % to 78.8 % with true-premise confirmation and arithmetic unchanged at 100 %. An ablation without the true-premise pairs shows the shortcut *always say no* (true facts confirmed: 0 %).

Run output

```
$ python 01_DPO_Loss_Core.py
======================================================================
USSE NOOSPHERE • SUBSYSTEM 04 · PART 1: DPO LOSS CORE
======================================================================
[unit] random logits: loss 0.5780, policy grad norm 0.0774, reference grad None: True

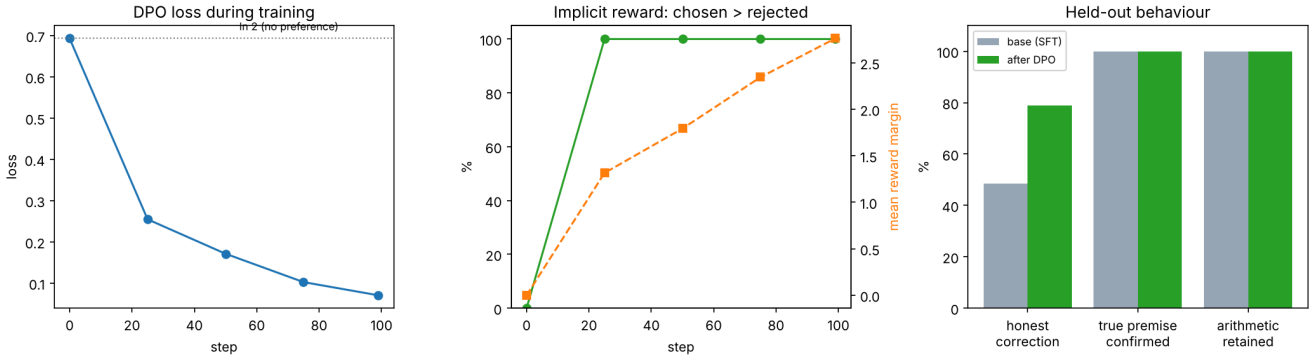
[AURA Lab] DPO on (honest > sycophantic) pairs, beta = 0.1
  step    0  loss 0.6931  chosen_win_rate +0.000  reward_margin +0.000  bt_probability +0.500
chosen_rewards +0.000  rejected_rewards +0.000
  step   25  loss 0.2548  chosen_win_rate +1.000  reward_margin +1.316  bt_probability +0.779
chosen_rewards -0.023  rejected_rewards -1.339
  step   50  loss 0.1714  chosen_win_rate +1.000  reward_margin +1.793  bt_probability +0.845
chosen_rewards -0.029  rejected_rewards -1.822
  step   75  loss 0.1026  chosen_win_rate +1.000  reward_margin +2.345  bt_probability +0.904
chosen_rewards -0.006  rejected_rewards -2.351
  step   99  loss 0.0707  chosen_win_rate +1.000  reward_margin +2.762  bt_probability +0.933
chosen_rewards -0.033  rejected_rewards -2.795

Gradient isolation (one real batch): policy tensors with grad 41/41, reference tensors with grad 0/41,
policy grad norm 0.798
Initial loss 0.6931 (ln 2 = 0.6931: policy = reference) -> final batch loss 0.0707

held-out metric                     base (SFT)  after DPO
honest correction of false premise %        48.5       78.8
true premise confirmed %                   100.0      100.0
arithmetic retained %                      100.0      100.0
perplexity on honest text                    1.5        1.6
mean answer length (words)                   7.0       14.0
verbose answers %                            0.0      100.0

[ablation] same run WITHOUT the true-premise pairs (only honest > sycophantic):
  honest 84.8%  true premise confirmed 0.0%  arithmetic 81.2%  -> the policy learns the shortcut 'always say
no'
```

Figures



Source code

```
"""
Mission 04 — Part 1: Systems Architect (Lead Engineer)
Direct Preference Optimization (DPO) Loss Core & Log-Likelihood Extraction

Starship Operational Unit: USSE Noosphere Bridge Crew
Sector: SEC-04-ORI (Betelgeuse & Orion Molecular Cloud Complex)
Subsystem 04: Plasma Ignition & Ethical Steering Core
"""

import json
import math
import os
import pathlib
from typing import Dict, Optional, Tuple, Union

import torch
import torch.nn as nn
import torch.nn.functional as F

try:
    import matplotlib.pyplot as plt
    HAVE_MATPLOTLIB = True
except ImportError:
    HAVE_MATPLOTLIB = False


def get_batch_logps(
    logits: torch.FloatTensor,
    labels: torch.LongTensor,
    label_pad_token_id: int = -100,
    average_log_prob: bool = False
) -> torch.FloatTensor:
    """
    Computes per-sequence log-probabilities of token labels under model logits.

    Args:
        logits: Tensor of shape (batch_size, seq_len, vocab_size)
        labels: Tensor of shape (batch_size, seq_len) with prompt/pad tokens set to label_pad_token_id
        label_pad_token_id: Token ID to ignore in loss computation
        average_log_prob: If True, returns mean log-prob per non-pad token (used in SimPO)

    Returns:
        Tensor of shape (batch_size,) containing sequence log-probabilities
    """
    if logits.shape[1] <= 1:
        raise ValueError("Sequence length must be > 1 to compute next-token logps.")

    # Shift so that tokens < n predict n
    shift_logits = logits[:, :-1, :].contiguous()
    shift_labels = labels[:, 1:].contiguous()

    # Create loss mask for non-padding tokens
    loss_mask = (shift_labels != label_pad_token_id)

    # Replace padding tokens with 0 for gather operation
    clean_labels = shift_labels.clone()
    clean_labels[~loss_mask] = 0

    # Calculate log-softmax over vocabulary
    log_probs = F.log_softmax(shift_logits, dim=-1)
```

```
    # Gather log-probabilities corresponding to true token labels
    per_token_logps = torch.gather(
        log_probs,
        dim=2,
        index=clean_labels.unsqueeze(2)
    ).squeeze(2)

    # Mask out padding tokens
    per_token_logps = per_token_logps * loss_mask.float()

    if average_log_prob:
        seq_lengths = loss_mask.sum(dim=-1).clamp(min=1)
        return per_token_logps.sum(dim=-1) / seq_lengths
    else:
        return per_token_logps.sum(dim=-1)


class DPOLoss(nn.Module):
    """
    Direct Preference Optimization (DPO) Loss Module (Rafailov et al., NeurIPS 2023).

    Optimizes policy directly from pairwise preferences without training an explicit reward model:
        L_DPO = -E_{(x, yw, yl)} [ log sigma( beta * ( log(pi(yw)/pi_ref(yw)) - log(pi(yl)/pi_ref(yl)) ) ) ]
    """

    def __init__(self, beta: float = 0.1, label_pad_token_id: int = -100):
        super().__init__()
        self.beta = beta
        self.label_pad_token_id = label_pad_token_id

    def forward(
        self,
        policy_chosen_logps: torch.FloatTensor,
        policy_rejected_logps: torch.FloatTensor,
        reference_chosen_logps: torch.FloatTensor,
        reference_rejected_logps: torch.FloatTensor
    ) -> Tuple[torch.FloatTensor, Dict[str, torch.FloatTensor]]:
        """
        Calculates DPO loss, implicit rewards, and preference margins.
        """
        # Calculate log-ratio differences: log(pi_theta(y|x) / pi_ref(y|x))
        chosen_logratios = policy_chosen_logps - reference_chosen_logps
        rejected_logratios = policy_rejected_logps - reference_rejected_logps

        # Implicit rewards derived analytically via Rafailov reparameterization:
        # r(x, y) = beta * log(pi_theta(y|x) / pi_ref(y|x))
        chosen_rewards = self.beta * chosen_logratios.detach()
        rejected_rewards = self.beta * rejected_logratios.detach()
        reward_margins = chosen_rewards - rejected_rewards

        # Compute Bradley-Terry logit: beta * (chosen_logratios - rejected_logratios)
        logits = self.beta * (chosen_logratios - rejected_logratios)

        # Numerically stable DPO loss: -log(sigmoid(logits))
        loss = -F.logsigmoid(logits).mean()

        # Metrics for monitoring training
        with torch.no_grad():
            chosen_win_rate = (chosen_rewards > rejected_rewards).float().mean()
            mean_margin = reward_margins.mean()
            bt_probs = torch.sigmoid(logits).mean()

        metrics = {
            "loss": loss.detach(),
            "chosen_win_rate": chosen_win_rate,
            "reward_margin": mean_margin,
            "bt_probability": bt_probs,
            "chosen_rewards": chosen_rewards.mean(),
            "rejected_rewards": rejected_rewards.mean()
        }

        return loss, metrics


def gradient_isolation_check(policy: nn.Module, reference: nn.Module, loss: torch.Tensor) -> Dict[str, float]:
    """After loss.backward(): every policy weight has a gradient, no reference weight does."""
    pol = [p for p in policy.parameters()]
    ref = [p for p in reference.parameters()]
    policy_with_grad = sum(p.grad is not None and p.grad.abs().sum().item() > 0 for p in pol)
```

```
    ref_with_grad = sum(p.grad is not None for p in ref)
    grad_norm = torch.sqrt(sum((p.grad ** 2).sum() for p in pol if p.grad is not None)).item()
    return {"policy_tensors": len(pol), "policy_tensors_with_grad": policy_with_grad,
            "reference_tensors": len(ref), "reference_tensors_with_grad": ref_with_grad,
            "policy_grad_norm": grad_norm}


def make_dpo_objective(policy: nn.Module, reference: nn.Module, beta: float = 0.1):
    """Closure used by aura_lab.train_preference: DPO on (chosen, rejected) example lists."""
    import aura_lab as L
    criterion = DPOLoss(beta=beta)

    def objective(chosen, rejected):
        pc = L.sequence_logps(policy, chosen, grad=True)
        pr = L.sequence_logps(policy, rejected, grad=True)
        rc = L.sequence_logps(reference, chosen)
        rr = L.sequence_logps(reference, rejected)
        return criterion(pc, pr, rc, rr)
    return objective


def train_dpo_on_aura(beta: float = 0.1, steps: int = 100, verbose: bool = True,
                      include_true_premises: bool = True, save: bool = True):
    """Real DPO run on the AURA Lab base model. Returns (policy, reference, report)."""
    import copy
    import aura_lab as L
    base = L.base_model(verbose=verbose)
    reference = L.clone_frozen(base)
    policy = copy.deepcopy(base)
    train_fp, held_fp = L.split_false_premises()
    pairs = L.preference_pairs(train_fp, include_true_premises=include_true_premises)

    before = L.evaluate(policy)
    before["perplexity"] = L.perplexity(policy)

    # gradient isolation check on one real batch, before any update
    objective = make_dpo_objective(policy, reference, beta)
    policy.train()
    loss0, _ = objective([c for c, _ in pairs[:16]], [r for _, r in pairs[:16]])
    loss0.backward()
    grad_check = gradient_isolation_check(policy, reference, loss0)
    policy.zero_grad(set_to_none=True)

    history = L.train_preference(policy, pairs, objective, steps=steps, verbose=verbose)
    after = L.evaluate(policy)
    after["perplexity"] = L.perplexity(policy)
    if save:
        L.CACHE.parent.mkdir(exist_ok=True)
        torch.save(policy.state_dict(), L.CACHE.parent / "aura_dpo_policy.pt")
    return policy, reference, {"beta": beta, "steps": steps, "train_pairs": len(pairs),
                               "held_out_prompts": len(held_fp), "initial_loss": loss0.item(),
                               "gradient_check": grad_check, "history": history,
                               "before": before, "after": after}


def save_dpo_artifacts(unit: Dict, report: Dict, output_dir: pathlib.Path):
    """Saves results/01_dpo_core.json and figures/01_dpo_training.png."""
    results_dir, figures_dir = output_dir / "results", output_dir / "figures"
    results_dir.mkdir(parents=True, exist_ok=True); figures_dir.mkdir(parents=True, exist_ok=True)
    with open(results_dir / "01_dpo_core.json", "w") as f:
        json.dump({"unit_check_random_logits": unit, "aura_dpo_training": report}, f, indent=2)
    if not HAVE_MATPLOTLIB:
        return
    h = report["history"]
    steps = [r["step"] for r in h]
    fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(14, 4))
    ax1.plot(steps, [r["loss"] for r in h], "o-", color="#1f77b4")
    ax1.axhline(math.log(2), color="grey", ls=":", lw=1); ax1.text(steps[-1] * 0.55, math.log(2) + 0.02, "ln 2 (no
preference)", fontsize=8)
    ax1.set_title("DPO loss during training"); ax1.set_xlabel("step"); ax1.set_ylabel("loss")
    ax2.plot(steps, [100 * r["chosen_win_rate"] for r in h], "o-", color="#2ca02c", label="batch reward accuracy")
    ax2.set_ylim(0, 105); ax2.set_title("Implicit reward: chosen > rejected"); ax2.set_xlabel("step");
ax2.set_ylabel("%")
    ax2b = ax2.twinx(); ax2b.plot(steps, [r["reward_margin"] for r in h], "s--", color="#ff7f0e", label="mean
margin")
    ax2b.set_ylabel("mean reward margin", color="#ff7f0e")
    b, a = report["before"], report["after"]
    keys = [("honest_correction_pct", "honest\ncorrection"), ("true_premise_confirmed_pct", "true
premise\nconfirmed"),
            ("arithmetic_accuracy_pct", "arithmetic\nretained")]
```

```
    x = range(len(keys))
    ax3.bar([i - 0.2 for i in x], [b[k] for k, _ in keys], 0.4, label="base (SFT)", color="#9aa7b5")
    ax3.bar([i + 0.2 for i in x], [a[k] for k, _ in keys], 0.4, label="after DPO", color="#2ca02c")
    ax3.set_xticks(list(x)); ax3.set_xticklabels([l for _, l in keys]); ax3.set_ylim(0, 110)
    ax3.set_title("Held-out behaviour"); ax3.set_ylabel("%"); ax3.legend(fontsize=8)
    plt.tight_layout(); plt.savefig(figures_dir / "01_dpo_training.png", dpi=150); plt.close()


def unit_check_random_logits() -> Dict[str, float]:
    """Quick shape / masking / stability check of get_batch_logps + DPOLoss on random logits."""
    torch.manual_seed(42)
    batch_size, seq_len, vocab_size = 4, 16, 128
    policy_logits_w = torch.randn(batch_size, seq_len, vocab_size, requires_grad=True)
    policy_logits_l = torch.randn(batch_size, seq_len, vocab_size, requires_grad=True)
    ref_logits_w = torch.randn(batch_size, seq_len, vocab_size)
    ref_logits_l = torch.randn(batch_size, seq_len, vocab_size)
    labels_w = torch.randint(0, vocab_size, (batch_size, seq_len)); labels_w[:, :4] = -100
    labels_l = torch.randint(0, vocab_size, (batch_size, seq_len)); labels_l[:, :4] = -100
    loss, metrics = DPOLoss(beta=0.1)(get_batch_logps(policy_logits_w, labels_w), get_batch_logps(policy_logits_l,
labels_l),
                                      get_batch_logps(ref_logits_w, labels_w), get_batch_logps(ref_logits_l,
labels_l))
    loss.backward()
    return {"loss": loss.item(), "bt_probability": metrics["bt_probability"].item(),
            "policy_logits_grad_norm": policy_logits_w.grad.norm().item(),
            "reference_logits_grad_is_none": ref_logits_w.grad is None}


def run_dpo_simulation(output_dir: Optional[pathlib.Path] = None):
    """Part 1: unit check on random logits, then a real DPO run on the AURA Lab model."""
    print("=" * 70)
    print("USSE NOOSPHERE • SUBSYSTEM 04 · PART 1: DPO LOSS CORE")
    print("=" * 70)
    unit = unit_check_random_logits()
    print(f"[unit] random logits: loss {unit['loss']:.4f}, policy grad norm {unit['policy_logits_grad_norm']:.4f}, "
          f"reference grad None: {unit['reference_logits_grad_is_none']}")

    print("\n[AURA Lab] DPO on (honest > sycophantic) pairs, beta = 0.1")
    _, _, rep = train_dpo_on_aura()
    g = rep["gradient_check"]
    print(f"\nGradient isolation (one real batch): policy tensors with grad
{g['policy_tensors_with_grad']}/{g['policy_tensors']}, "
          f"reference tensors with grad {g['reference_tensors_with_grad']}/{g['reference_tensors']}, policy grad
norm {g['policy_grad_norm']:.3f}")
    print(f"Initial loss {rep['initial_loss']:.4f} (ln 2 = {math.log(2):.4f}: policy = reference) -> "
          f"final batch loss {rep['history'][-1]['loss']:.4f}")
    b, a = rep["before"], rep["after"]
    print(f"\n{'held-out metric':<34} {'base (SFT)':>11} {'after DPO':>10}")
    for k, lab in [("honest_correction_pct", "honest correction of false premise %"),
                   ("true_premise_confirmed_pct", "true premise confirmed %"),
                   ("arithmetic_accuracy_pct", "arithmetic retained %"),
                   ("perplexity", "perplexity on honest text"),
                   ("mean_answer_words", "mean answer length (words)"),
                   ("verbose_answer_pct", "verbose answers %")]:
        print(f"{lab:<34} {b[k]:>11.1f} {a[k]:>10.1f}")

    print("\n[ablation] same run WITHOUT the true-premise pairs (only honest > sycophantic):")
    _, _, abl = train_dpo_on_aura(verbose=False, include_true_premises=False, save=False)
    x = abl["after"]
    print(f"  honest {x['honest_correction_pct']:.1f}%  true premise confirmed
{x['true_premise_confirmed_pct']:.1f}%  "
          f"arithmetic {x['arithmetic_accuracy_pct']:.1f}%  -> the policy learns the shortcut 'always say no'")
    rep["ablation_without_true_premise_pairs"] = {k: v for k, v in abl.items() if k != "history"}
    print("=" * 70)
    if output_dir:
        save_dpo_artifacts(unit, rep, output_dir)


if __name__ == "__main__":
    current_dir = pathlib.Path(__file__).resolve().parent
    run_dpo_simulation(output_dir=current_dir)
```